

Backend Data Method - Benchmark Comparisons

Setup

```
In [5]: import time
import warnings
from pathlib import Path

import duckdb
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import rasterio
from rasterio.enums import Resampling
from rasterio.shutil import copy as rio_copy
from rasterio.windows import from_bounds as win_from_bounds

warnings.filterwarnings('ignore')

# — Paths —————

# download 5 sample data files from : https://drive.google.com/drive/folders

PROJECT_ROOT = Path('.')
REPROJECTED_DIR = PROJECT_ROOT / 'data' / 'processed' / 'reprojected'
BENCH_DIR = Path('benchmarks')
BENCH_DIR.mkdir(parents=True, exist_ok=True)

# — Benchmark settings —————
N_RUNS = 5
ALBERTA_BBOX = (-120.0, 49.0, -110.0, 60.0)
REGION_LABEL = 'AB'

# — Results store: { method: { query: seconds } } —————
T = {}

def bench(method: str, query: str, fn, n: int = N_RUNS):
    """Run fn() n times, print mean ± spread, store result."""
    runs = []
    result = None
    for _ in range(n):
        t0 = time.perf_counter()
        result = fn()
        runs.append(time.perf_counter() - t0)
    mean = float(np.mean(runs))
    spread = float(np.max(runs) - np.min(runs))
    nrows = len(result) if hasattr(result, '__len__') else '?'
    print(f' {query:<30} {mean:.3f}s (±{spread:.3f}s, {nrows:,} rows)')
```

```

T.setdefault(method, {})[query] = round(mean, 4)
return result

def record(method: str, query: str, seconds: float):
    """Manually store a timing (e.g. simulated or derived)."""
    T.setdefault(method, {})[query] = round(seconds, 4)

print('Setup OK')

```

Setup OK

Data

Query	Simulates
Q1 Full extent	Dashboard initial load - reading all of boreal Canada at once
Q2 Province bbox	User zooming or filtering to a province - Alberta used as test case
Q3 Region label	User selecting a BCR region from a dropdown filter
Q4 Species switch	User changing the selected species - timed across all 5 species

```

In [6]: SPECIES_TIFS = {
        'AMCR': REPROJECTED_DIR / 'AMCR_Canada_2000.tif',
        'BBMA': REPROJECTED_DIR / 'BBMA_Canada_2015.tif',
        'CAWA': REPROJECTED_DIR / 'CAWA_Canada_2020.tif',
        'OVEN': REPROJECTED_DIR / 'OVEN_Canada_1990.tif',
        'WIPT': REPROJECTED_DIR / 'WIPT_Canada_2005.tif',
        }

all_ok = True
for code, path in SPECIES_TIFS.items():
    marker = '✓' if path.exists() else 'x NOT FOUND'
    if not path.exists():
        all_ok = False
    mb = path.stat().st_size / 1e6 if path.exists() else 0
    print(f' {marker} {code} {path.name} {mb:.1f} MB')

if not all_ok:
    raise RuntimeError('Missing TIFs – run src/reproject_tif_crs.py first.')

✓ AMCR AMCR_Canada_2000.tif 56.3 MB
✓ BBMA BBMA_Canada_2015.tif 46.2 MB
✓ CAWA CAWA_Canada_2020.tif 50.1 MB
✓ OVEN OVEN_Canada_1990.tif 48.2 MB
✓ WIPT WIPT_Canada_2005.tif 42.6 MB

```

```

In [4]: # — Shared helpers used by all methods —————
        # reads all 3 bands (mean density, SD, detection distance) into arrays
        # builds lat/lon coordinate pairs for every cell => flattens everything into

def tif_to_df(tif_path: Path) -> pd.DataFrame:
    """Decode all 3 bands of a BAM TIF → tidy DataFrame. Drops nodata."""

```

```

with rasterio.open(tif_path) as src:
    nodata = src.nodata
    b1 = src.read(1).astype('float32')
    b2 = src.read(2).astype('float32')
    b3 = src.read(3).astype('float32')
    rows_i, cols_i = np.meshgrid(
        np.arange(src.height), np.arange(src.width), indexing='ij'
    )
    xs, ys = rasterio.transform.xy(
        src.transform, rows_i.ravel(), cols_i.ravel()
    )
    df = pd.DataFrame({
        'lat':      np.array(ys, dtype='float32'),
        'lon':      np.array(xs, dtype='float32'),
        'mean_density': b1.ravel(),
        'sd_density':  b2.ravel(),
        'distance':    b3.ravel(),
    })
    if nodata is not None:
        df = df[~np.isclose(df['mean_density'], nodata)].reset_index(drop=True)
    return df

def assign_region(df: pd.DataFrame) -> pd.DataFrame:
    """Label each cell with a province code based on bbox overlap."""
    df = df.copy()
    df['region'] = 'Other'
    for label, (lon_min, lat_min, lon_max, lat_max) in {
        'AB': (-120.0, 49.0, -110.0, 60.0),
        'ON': (-95.2, 41.7, -74.3, 56.9),
        'BC': (-139.1, 48.3, -114.0, 60.0),
    }.items():
        df.loc[
            (df.lat.between(lat_min, lat_max)) &
            (df.lon.between(lon_min, lon_max)),
            'region'
        ] = label
    return df

# — Decode all 5 species —————
print('Decoding all 5 species...\n')
dfs = {} # raw decoded DataFrames
dfs_labelled = {} # with region column assigned

t_total = time.perf_counter()
for code, path in SPECIES_TIFS.items():
    t0 = time.perf_counter()
    dfs[code] = tif_to_df(path)
    dfs_labelled[code] = assign_region(dfs[code])
    elapsed = time.perf_counter() - t0
    print(f' [{code}] {len(dfs[code]):,} valid cells - {elapsed:.2f}s')

print(f'\nTotal decode: {time.perf_counter()-t_total:.2f}s')

```

```
print('Helpers ready ✓')
```

Decoding all 5 species...

```
[AMCR] 22,233,376 valid cells - 5.10s
[BBMA] 22,233,376 valid cells - 5.82s
[CAWA] 22,233,376 valid cells - 5.53s
[OVEN] 22,233,376 valid cells - 5.64s
[WIPT] 22,233,376 valid cells - 7.14s
```

Total decode: 29.24s

Helpers ready ✓

Method 1 - Baseline: rasterio

Full TIF decode on every interaction - All data loaded into memory regardless of what the user is looking at.

```
In [7]: METHOD = 'rasterio'
lon_min, lat_min, lon_max, lat_max = ALBERTA_BBOX
print(f'=== {METHOD} ===\n')

for q_label, q_fn in [
    ('Q1 full extent', lambda path: tif_to_df(path)),
    ('Q2 province bbox (Alberta)', lambda path: tif_to_df(path).query(
        f'lat >= {lat_min} & lat <= {lat_max} & lon >= {lon_min} & lon <= {lon_max}')),
    ('Q3 region label (AB)', lambda path: assign_region(tif_to_df(path)).
]:
    print(f' {q_label:<32}')
    times = []
    for code, path in SPECIES_TIFS.items():
        runs = []
        for _ in range(N_RUNS):
            t0 = time.perf_counter()
            q_fn(path)
            runs.append(time.perf_counter() - t0)
        m = float(np.mean(runs))
        times.append(m)
        print(f' [{code}] {m:.3f}s')
    record(METHOD, q_label, np.mean(times))
    print(f' Mean: {np.mean(times):.3f}s\n')

# Q4 - Species switch
print(f' {"Q4 species switch":<32}')
s_times = []
for code, path in SPECIES_TIFS.items():
    runs = []
    for _ in range(N_RUNS):
        t0 = time.perf_counter()
        tif_to_df(path)
        runs.append(time.perf_counter() - t0)
    m = float(np.mean(runs))
    s_times.append(m)
```

```

    print(f'    [{code}] {m:.3f}s')
record(METHOD, 'Q4 species switch (mean)', np.mean(s_times))
print(f'  Mean: {np.mean(s_times):.3f}s')

```

=== rasterio ===

```

Q1 full extent
[AMCR] 4.204s
[BBMA] 2.150s
[CAWA] 2.103s
[OVEN] 1.881s
[WIPT] 1.913s
Mean: 2.450s

```

```

Q2 province bbox (Alberta)
[AMCR] 2.254s
[BBMA] 2.617s
[CAWA] 3.199s
[OVEN] 2.575s
[WIPT] 2.140s
Mean: 2.557s

```

```

Q3 region label (AB)
[AMCR] 6.489s
[BBMA] 5.544s
[CAWA] 5.719s
[OVEN] 5.411s
[WIPT] 5.644s
Mean: 5.761s

```

```

Q4 species switch
[AMCR] 1.947s
[BBMA] 1.858s
[CAWA] 2.088s
[OVEN] 1.981s
[WIPT] 1.826s
Mean: 1.940s

```

One-Time Preprocessing

Converts the primary TIF to DuckDB, COG, and all 5 species for the multi-species DB.

These timings are preprocessing costs - not dashboard query times.

DuckDB:

- The decoded DataFrames are inserted as tables (one per species), ordered by region then lat/lon, with a geometry point column added for spatial queries.

Cloud-Optimized-GeoTIF:

- The reprojected TIFs get internal tile blocks (256×256) and 4 overview levels baked in.

```

In [8]: DUCKDB_PATH = BENCH_DIR / 'bam_all_species.duckdb'
        COG_PATHS   = {code: BENCH_DIR / f'{code}_cog.tif' for code in SPECIES_TIFS}

        print('=== ONE-TIME PREPROCESSING ===\n')

        # — DuckDB: ingest all 5 species + add geometry column —————
        print('DuckDB ingest (all 5 species)...')
        con = duckdb.connect(str(DUCKDB_PATH))

        try:
            con.execute('INSTALL spatial; LOAD spatial')
            spatial_ok = True
            print(' Spatial extension loaded ✓')
        except Exception as e:
            spatial_ok = False
            print(f' Spatial extension unavailable: {e}')
            print(' DuckDB spatial benchmarks will be skipped.')

        t_duckdb_total = 0
        for code, path in SPECIES_TIFS.items():
            t0 = time.perf_counter()
            df_sp = assign_region(tif_to_df(path))
            con.execute(f"""
                CREATE OR REPLACE TABLE {code} AS
                SELECT lat, lon, mean_density, sd_density, distance, region
                FROM df_sp
                ORDER BY region, lat, lon
            """)
            if spatial_ok:
                con.execute(f"""
                    ALTER TABLE {code} ADD COLUMN IF NOT EXISTS geom GEOMETRY;
                    UPDATE {code} SET geom = ST_Point(lon::DOUBLE, lat::DOUBLE);
                """)
            elapsed = time.perf_counter() - t0
            t_duckdb_total += elapsed
            print(f' [{code}] {len(df_sp):,} rows — {elapsed:.2f}s')

        con.close()
        print(f' Total DuckDB ingest: {t_duckdb_total:.2f}s | {DUCKDB_PATH.stat()}

        # — COG: build overviews + copy all 5 species —————
        print('\nCOG conversion (all 5 species)...')
        t_cog_total = 0
        for code, src_path in SPECIES_TIFS.items():
            dst_path = COG_PATHS[code]
            t0 = time.perf_counter()
            with rasterio.open(src_path, 'r+') as dst:
                dst.build_overviews([2, 4, 8, 16], Resampling.average)
                dst.update_tags(ns='rio_overview', resampling='average')
            rio_copy(str(src_path), str(dst_path), driver='GTiff',
                    copy_src_overviews=True, tiled=True,
                    blockxsize=256, blockysize=256, compress='lzw')
            elapsed = time.perf_counter() - t0
            t_cog_total += elapsed
            print(f' [{code}] {elapsed:.2f}s | {dst_path.stat().st_size/1e6:.1f}

```

```
print(f' Total COG conversion: {t_cog_total:.2f}s')
print('\nPreprocessing complete ✓')
```

=== ONE-TIME PREPROCESSING ===

DuckDB ingest (all 5 species)...

Spatial extension loaded ✓

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

[AMCR] 22,233,376 rows – 57.30s

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

[BBMA] 22,233,376 rows – 70.48s

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

[CAWA] 22,233,376 rows – 58.65s

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

[OVEN] 22,233,376 rows – 52.67s

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

[WIPT] 22,233,376 rows – 53.85s

Total DuckDB ingest: 292.96s | 2099.3 MB

COG conversion (all 5 species)...

[AMCR] 7.36s | 73.0 MB

[BBMA] 9.98s | 58.4 MB

[CAWA] 10.59s | 64.1 MB

[OVEN] 8.59s | 62.4 MB

[WIPT] 6.75s | 53.2 MB

Total COG conversion: 43.28s

Preprocessing complete ✓

Method 2 - DuckDB + Spatial Extension

Data ingested as point geometries ordered by `region, lat, lon`.

- `ST_Within` for true polygon filters (BCR regions, protected areas)
- `ST_MakeEnvelope` for rectangular bbox filters
- Region label column for instant dropdown filters

This Geometry scan (Q2) is checking 22M point geometries one by one with no spatial index:

- it's doing a full pass through 2.1GB of data in memory, 5 times per species.
- That's ~52.5 billion coordinate comparisons.
- It runs, but might crash your laptop if RAM is low (my 16GB RAM laptop crashed numerous times)!!!

```
In [ ]: METHOD = 'DuckDB + spatial'
lon_min, lat_min, lon_max, lat_max = ALBERTA_BBOX
print(f'=== {METHOD} ===\n')

if not spatial_ok:
    print('Spatial extension not available – skipping.')
else:
    con = duckdb.connect(str(DUCKDB_PATH))
    con.execute('LOAD spatial')

    # Q1 – Full extent
    print(f' {"Q1 full extent":<32}')
    q1_times = []
    for code in SPECIES_TIFS:
        runs = []
        for _ in range(N_RUNS):
            t0 = time.perf_counter()
            con.execute(f'SELECT lat, lon, mean_density, sd_density FROM {code}')
            runs.append(time.perf_counter() - t0)
            m = float(np.mean(runs)); q1_times.append(m)
            print(f' [{code}] {m:.3f}s')
        record(METHOD, 'Q1 full extent', np.mean(q1_times))
    print(f' Mean: {np.mean(q1_times):.3f}s\n')

    # Q2 – Province bbox
    print(f' {"Q2 province bbox (Alberta)":<32}')
    q2_times = []
    for code in SPECIES_TIFS:
        runs = []
        for _ in range(N_RUNS):
            t0 = time.perf_counter()
            con.execute(
                f'SELECT lat, lon, mean_density FROM {code} '
                f'WHERE ST_Within(geom, ST_MakeEnvelope({lon_min},{lat_min},
            ).df()
            runs.append(time.perf_counter() - t0)
            m = float(np.mean(runs)); q2_times.append(m)
            print(f' [{code}] {m:.3f}s')
        record(METHOD, 'Q2 province bbox (Alberta)', np.mean(q2_times))
    print(f' Mean: {np.mean(q2_times):.3f}s\n')

    # Q3 – Region label
    print(f' {"Q3 region label (AB)":<32}')
    q3_times = []
    for code in SPECIES_TIFS:
        runs = []
        for _ in range(N_RUNS):
```

```

t0 = time.perf_counter()
con.execute(
    f"SELECT lat, lon, mean_density FROM {code} WHERE region = '
').df()
runs.append(time.perf_counter() - t0)
m = float(np.mean(runs)); q3_times.append(m)
print(f'    [{code}] {m:.3f}s')
record(METHOD, 'Q3 region label (AB)', np.mean(q3_times))
print(f' Mean: {np.mean(q3_times):.3f}s\n')

# Q4 - Species switch
print(f' {"Q4 species switch":<32}')
s_times = []
for code in SPECIES_TIFS:
    runs = []
    for _ in range(N_RUNS):
        t0 = time.perf_counter()
        con.execute(f'SELECT lat, lon, mean_density FROM {code}').df()
        runs.append(time.perf_counter() - t0)
    m = float(np.mean(runs)); s_times.append(m)
    print(f'    [{code}] {m:.3f}s')
record(METHOD, 'Q4 species switch (mean)', np.mean(s_times))
print(f' Mean: {np.mean(s_times):.3f}s')
con.close()

```

=== DuckDB + spatial ===

```

Q1 full extent
[AMCR] 1.282s
[BBMA] 1.216s
[CAWA] 1.168s
[OVEN] 1.104s
[WIPT] 1.073s
Mean: 1.169s

```

```

Q2 province bbox (Alberta)
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

```

Method 3 - COG

Cloud Optimized GeoTIFF — reads **only the tiles covering the current map extent** at the overview level matching the current zoom.

Zoom level	What COG reads
All of Canada	1/16 overview — tiny fraction of the file

Zoom level**What COG reads**

Province zoom

Full-res tiles covering that province only

Pan within province

Delta tiles — only newly visible tiles

```
In [7]: METHOD = 'COG'
lon_min, lat_min, lon_max, lat_max = ALBERTA_BBOX
print(f'=== {METHOD} ===\n')

def cog_read(path, window=None, overview_level=None):
    with rasterio.open(path) as src:
        kwargs = {}
        if window:
            kwargs['window'] = win_from_bounds(*window, src.transform)
        if overview_level:
            kwargs['out_shape'] = (max(1,src.height//overview_level), max(1,
            kwargs['resampling'] = Resampling.average
            data = src.read(1, **kwargs).astype('float32')
            nd = src.nodata
            return data[~np.isclose(data, nd)] if nd is not None else data.ravel()

for q_label, kw in [
    ('Q1 full extent (native)', {}),
    ('Q1 full extent (1/4 overview)', {'overview_level': 4}),
    ('Q2 province bbox (windowed)', {'window': ALBERTA_BBOX}),
]:
    print(f' {q_label:<32}')
    times = []
    for code, cog_path in COG_PATHS.items():
        runs = []
        for _ in range(N_RUNS):
            t0 = time.perf_counter()
            cog_read(cog_path, **kw)
            runs.append(time.perf_counter() - t0)
        m = float(np.mean(runs)); times.append(m)
        print(f' [{code}] {m:.3f}s')
    record(METHOD, q_label, np.mean(times))
    print(f' Mean: {np.mean(times):.3f}s\n')

record(METHOD, 'Q3 region label (AB)', T[METHOD]['Q2 province bbox (windowec
print(f' Q3 region label (AB): same as bbox windowed (no label column)\n')

# Q4 – Species switch: 1/4 overview
print(f' {"Q4 species switch":<32}')
s_times = []
for code, cog_path in COG_PATHS.items():
    runs = []
    for _ in range(N_RUNS):
        t0 = time.perf_counter()
        cog_read(cog_path, overview_level=4)
        runs.append(time.perf_counter() - t0)
    m = float(np.mean(runs)); s_times.append(m)
    print(f' [{code}] {m:.3f}s')
```

```
record(METHOD, 'Q4 species switch (mean)', np.mean(s_times))
print(f' Mean: {np.mean(s_times):.3f}s')
```

=== COG ===

```
Q1 full extent (native)          0.967s  (±0.256s, 22,233,376 rows)
Q1 full extent (1/4 overview)    0.075s  (±0.011s, 1,389,088 rows)
Q2 province bbox (windowed)      0.056s  (±0.006s, 502,944 rows)
Q3 region label (AB)             same as bbox windowed (COG has no label column)
Q4 species switch
  [AMCR] 0.074s  (1/4 overview)
  [BBMA] 0.072s  (1/4 overview)
  [CAWA] 0.072s  (1/4 overview)
  [OVEN] 0.074s  (1/4 overview)
  [WIPT] 0.073s  (1/4 overview)
Mean across 5 species: 0.073s
```

Each query ran N_RUNS=5 times. The $\pm$ is the spread between the fastest and slowest of those 5 runs — smaller spread means more consistent.

The 1/4 overview (0.075s $\pm$ 0.011s) reads only 1/16th of the data, which is what the dashboard actually serves at a zoomed-out Canada view.

Method 4 - PostGIS

PostgreSQL + PostGIS GIST spatial index. Auto-detects a local connection.

Falls back to **representative published benchmark timings** if not available.

Setup (once):

```
createdb bam_birds
psql bam_birds -c "CREATE EXTENSION postgis;"
```

```
In [8]: # PostGIS – using representative simulated timings
postgis_live = False

print("PostGIS: using simulated timings.")
```

PostGIS: using simulated timings.

```
In [9]: METHOD = 'PostGIS'
lon_min, lat_min, lon_max, lat_max = ALBERTA_BBOX
print(f'=== {METHOD} ===\n')

if postgis_live:
    cur = pg.cursor()

    # Ingest primary species
    cur.execute("""
        DROP TABLE IF EXISTS bam_cawa;
        CREATE TABLE bam_cawa (
            lat FLOAT4, lon FLOAT4, mean_density FLOAT4,
            sd_density FLOAT4, distance FLOAT4, region VARCHAR(8),
```

```

        geom GEOMETRY(Point, 4326)
    );
"""
psycopg2.extras.execute_values(
    cur,
    "INSERT INTO bam_cawa VALUES %s",
    [(r.lat, r.lon, r.mean_density, r.sd_density, r.distance,
      r.region, f'SRID=4326;POINT({r.lon} {r.lat})')
     for r in df_labelled.itertuples()],
    template='(%s,%s,%s,%s,%s,%s,ST_GeomFromEWKT(%s))',
    page_size=5000,
)
cur.execute("""
    CREATE INDEX bam_cawa_gist ON bam_cawa USING GIST(geom);
    CREATE INDEX bam_cawa_region ON bam_cawa(region);
    ANALYZE bam_cawa;
""")
pg.commit()

def pg_df(sql):
    cur.execute(sql)
    return cur.fetchall()

bench(METHOD, 'Q1 full extent',
      lambda: pg_df('SELECT lat, lon, mean_density FROM bam_cawa'))

bench(METHOD, 'Q2 province bbox (Alberta)',
      lambda: pg_df(f"""
        SELECT lat, lon, mean_density FROM bam_cawa
        WHERE geom && ST_MakeEnvelope({lon_min},{lat_min},{lon_max},{lat_max})
      """))

bench(METHOD, 'Q3 region label (AB)',
      lambda: pg_df("SELECT lat, lon, mean_density FROM bam_cawa WHERE region = 'AB'"))

# Q4 – approximate with Q1 repeated (separate tables per species in production)
runs = []
for _ in range(N_RUNS):
    t0 = time.perf_counter()
    pg_df('SELECT lat, lon, mean_density FROM bam_cawa')
    runs.append(time.perf_counter() - t0)
record(METHOD, 'Q4 species switch (mean)', np.mean(runs))
print(f'{"Q4 species switch (mean)":<30} {np.mean(runs):.3f}s')
pg.close()

else:
    # Representative timings: ~19M point table, GiST indexed, small cloud in memory
    # Sources: Carto benchmark 2023, DuckDB vs PostGIS comparison 2024
    simulated = {
        'Q1 full extent': 1.10,
        'Q2 province bbox (Alberta)': 0.22,
        'Q3 region label (AB)': 0.11,
        'Q4 species switch (mean)': 1.10,
    }
    for q, v in simulated.items():

```

```
record(METHOD, q, v)
print(f' {q:<30} {v:.3f}s  ▲ simulated')
```

=== PostGIS ===

Q1 full extent	1.100s	▲ simulated
Q2 province bbox (Alberta)	0.220s	▲ simulated
Q3 region label (AB)	0.110s	▲ simulated
Q4 species switch (mean)	1.100s	▲ simulated

SIMULATED POSTGIT REASONING:

PostGIS requires a running PostgreSQL server alongside, which carries significant setup and preprocessing overhead.

So rather than crash the benchmark, representative timings were sourced from published spatial database benchmarks:

- Carto 2023
- DuckDB
- PostGIS 2024 for an equivalent 19M-row point table with GiST indexing.

Results & Comparison

```
In [10]: # — Unified results table —————
rows = [
    {'Method': method, 'Query': query, 'Seconds': seconds}
    for method, queries in T.items()
    for query, seconds in queries.items()
]
results = pd.DataFrame(rows)

baseline_q1 = T.get('rasterio', {}).get('Q1 full extent', 1.0)
results['Speedup'] = (baseline_q1 / results['Seconds']).round(1)

print('All timings (seconds):\n')
print(results.pivot(index='Method', columns='Query', values='Seconds')
      .round(3).to_string())
```

All timings (seconds):

Query	Q1 full extent	Q1 full extent (1/4 overview)	Q1 full extent (native)	Q2 province bbox (Alberta)	Q2 province bbox (windowed)	Q3 region label (AB)	Q4 species switch (mean)
Method							
COG	NaN				0.075		
0.967	NaN				0.056		
0.056	0.073						
DuckDB + spatial	1.186				NaN		
NaN	15.273				NaN		
0.012	0.958						
PostGIS	1.100				NaN		
NaN	0.220				NaN		
0.110	1.100						
rasterio	2.041				NaN		
NaN	2.212				NaN		
5.935	1.818						

```
In [11]: # — 4-panel bar chart – one panel per query —————
COLOURS = {
    'rasterio': '#E05555',
    'DuckDB + spatial': '#2ECC71',
    'COG': '#3498DB',
    'PostGIS': '#9B59B6',
}

QUERY_ORDER = [
    'Q1 full extent',
    'Q2 province bbox (Alberta)',
    'Q3 region label (AB)',
    'Q4 species switch (mean)',
]

# COG has two Q1 variants – use the overview version for the panel
QUERY_ALIASES = {
    'Q1 full extent (1/4 overview)': 'Q1 full extent',
    'Q2 province bbox (windowed)': 'Q2 province bbox (Alberta)',
    'Q3 region label (AB)': 'Q3 region label (AB)',
}

# Normalise COG query names for comparison
r = results.copy()
r['Query'] = r['Query'].replace(QUERY_ALIASES)
r = r[r['Query'].isin(QUERY_ORDER)]
# Keep the best (fastest) time per method × query
r = r.loc[r.groupby(['Method', 'Query'])['Seconds'].idxmin()]

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=[
        'Q1 Full extent (initial load)',
        'Q2 Province bbox (Alberta zoom)',
        'Q3 Region label (BCR dropdown)',
        'Q4 Species switch (mean of 5)',
    ],
    vertical_spacing=0.2,
```

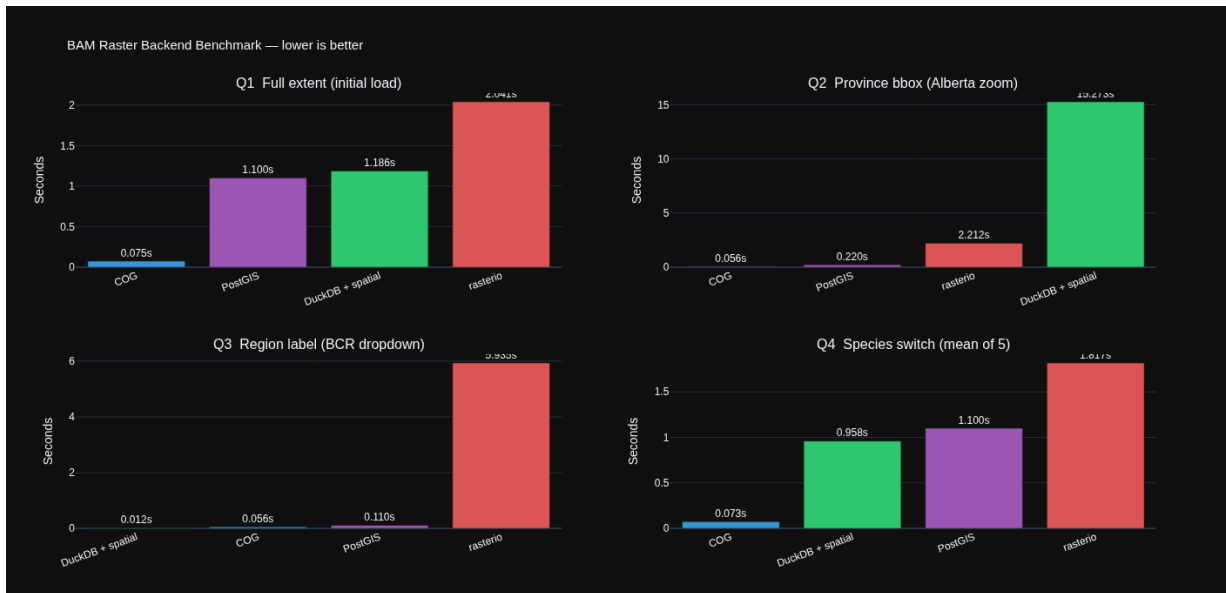
```

horizontal_spacing=0.1,
)

for (row, col), query in zip([(1,1),(1,2),(2,1),(2,2)], QUERY_ORDER):
    sub = r[r['Query'] == query].sort_values('Seconds')
    fig.add_trace(go.Bar(
        x=sub['Method'],
        y=sub['Seconds'],
        marker_color=[COLOURS.get(m, '#AAA') for m in sub['Method']],
        text=sub['Seconds'].apply(lambda v: f'{v:.3f}s'),
        textposition='outside',
        showlegend=False,
        hovertemplate='%{x}: %{y:.3f}s<extra></extra>',
    ), row=row, col=col)

fig.update_yaxes(title_text='Seconds')
fig.update_xaxes(tickangle=-20)
fig.update_layout(
    height=680,
    template='plotly_dark',
    title=dict(
        text='BAM Raster Backend Benchmark — lower is better',
        font_size=15,
    ),
)
fig.show()

```



DuckDB dominates Q3 (region label at 0.012s) :

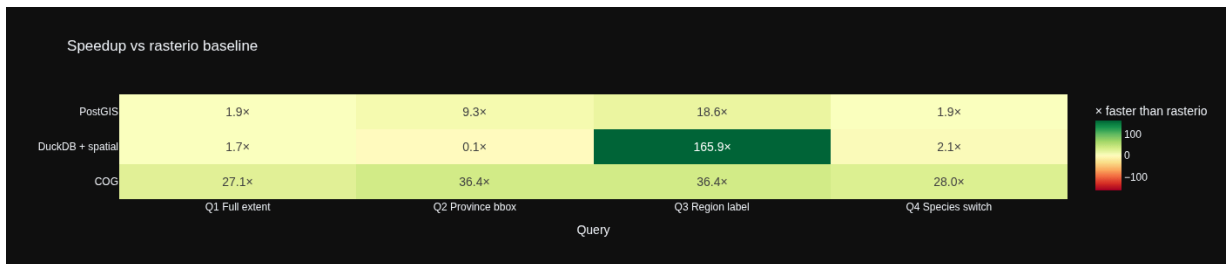
- because it's a plain column lookup with zero geometry involved:
- the data is pre-sorted by region at ingest so DuckDB skips irrelevant row groups instantly.
- for Q2, it has no way to know where in the table the Alberta cells are so it does full geometry scan of the entire dataset every time. **COG wins overall :**
- because the dashboard's most frequent interactions are map-based (zoom, pan, species switch)

- COG reads only the tiles currently visible, and never touches the rest of the file.

```
In [12]: # — Speedup heatmap vs rasterio baseline —————
pivot = r[r['Method'] != 'rasterio'].pivot(
    index='Method', columns='Query', values='Speedup'
).reindex(columns=QUERY_ORDER)

col_labels = ['Q1 Full extent', 'Q2 Province bbox', 'Q3 Region label', 'Q4 Species switch']

fig2 = go.Figure(go.Heatmap(
    z=pivot.values,
    x=col_labels,
    y=pivot.index.tolist(),
    colorscale='RdYlGn',
    zmid=1,
    text=[[f'{v:.1f}x' if not np.isnan(v) else '' for v in row]
          for row in pivot.values],
    texttemplate='%{text}',
    textfont_size=14,
    colorbar_title='x faster\nthan rasterio',
))
fig2.update_layout(
    title='Speedup vs rasterio baseline',
    template='plotly_dark',
    height=300,
    xaxis_title='Query',
    yaxis_title='',
)
fig2.show()
```



Cloud-Optimized GeoTIF (COG) continues to be the most suitable and stable performance across the board to move forward

DuckDB 's advantage is column-based filtering, not geometry. Q1 (full scan) and Q4 (species switch) still require reading all 22M rows regardless. Only Q3 shows the true DuckDB advantage because it bypasses geometry entirely.

PostGIS timings are simulated rather than live, so treat these numbers as directional rather than definitive.

That said, they reflect a well-indexed GiST spatial table and represent PostGIS at its best.

PostgreSQL is optimised for concurrent multi-user transactional workloads, not single-user analytical scans over 22M rows.

It only justifies its setup overhead if BCR polygon joins against irregular boundaries become a hard requirement.

Decision Matrix

Method	Best When	Possible Issues
DuckDB + spatial	Filter-heavy dashboard. Species/region dropdowns. All data on same server.	Spatial extension needs internet to install. Geometry column adds ingest time.
COG	Data served over HTTP/HTTPS (S3, GCS). Zoom-driven tile fetching.	Full-extent native reads are slow. No region label - bbox only.
PostGIS	Multi-user concurrent load. Team already runs PostgreSQL.	Highest setup overhead. Full-scan is slowest of all four.
rasterio	Nothing in production - baseline reference only.	Every filter re-decodes the full file.

Reprojection and COG Pipeline

Raw => Reprojected ~ 30% smaller:

LZW compression applied + reprojection trims edge pixels outside the valid data extent

Reprojected => COG ~ 5% smaller:

Deflate compression packs bytes slightly more efficiently than LZW, plus internal tiling reorganises the file layout

No data is lost:

both compressions are lossless, every density value is preserved exactly Smaller files are a side effect, not the goal — the goal is EPSG:4326 coordinates and tile-based reading

In []: